

СУ “Св. Климент Охридски”, ФМИ
курс - Системи за паралелна обработка

Hello-WaTor

Мулти-нишкова симулация на хишници и плячка в
торовиден свят

Цветомир Стайков
1MI0800469
Компютърни науки
3 курс 2 група

2026

Съдържание

Въведение.....	3
Увод.....	3
Реализация.....	3
Технологии.....	3
Машины.....	3
Симулация.....	4
Структура на проекта.....	4
Агенти в симулацията.....	5
Риба.....	5
Акула.....	6
Алгоритъм.....	6
Сектори.....	6
Синхронизация.....	7
Случайности.....	7
Параметри.....	8
Визуализация.....	8
Резултати.....	8
Източници.....	8

Въведение

Увод

Този документ представя симулация на хищници и плячка по модела на A. K. Dewdney представена в статията от 1984 "Computer Recreations: Sharks and fish wage an ecological war on the toroidal planet Wa-Tor". Ние ще наричаме плячката - риби, а хищниците - акули. Разработен е проект, симулиращ правилата на симулацията. Данните се обработват с SPMD (Single program, multiple data) модел чрез нишки. Представени са данните от изпълнения и анализ върху тях.

Реализация

Технологии

Кода на симулацията е написан на C++ 20 с компилатор *g++ 15.2.1*.

Допълнителни библиотеки са SFML за визуализация, STL за стандартни команди.

Графиките са създадени от Python 3.14 скриптове с Altair

Машини

Кода е писан и тестван на лаптоп Thinkpad L13 Yoga с Linux 7.0.9 Fedora 43 64-bit, но финалните резултати са на машина със следните спецификации:

Architecture:	x86_64
CPU op-mode(s):	32-bit, 64-bit
Byte Order:	Little Endian
CPU(s):	32
On-line CPU(s) list:	0-31
Thread(s) per core:	2
Core(s) per socket:	8
Socket(s):	2
NUMA node(s):	2
Vendor ID:	GenuineIntel
CPU family:	6
Model:	45
Model name:	Intel(R) Xeon(R) CPU E5-2660 0 @ 2.20GHz
Stepping:	7

CPU MHz:	3000.000
CPU max MHz:	3000.0000
CPU min MHz:	1200.0000
BogoMIPS:	4389.50
Virtualization:	VT-x
L1d cache:	32K
L1i cache:	32K
L2 cache:	256K
L3 cache:	20480K
NUMA node0 CPU(s):	0-7,16-23
NUMA node1 CPU(s):	8-15,24-31

Симулация

Симулирания терен представлява двуизмерно поле от клетки. Всяка клетка представлява риба (зелен цвят), акула (червен цвят) и празно място (черен цвят). Времето се измерва в единицата хронони (chronons), като често ще ги наричаме стъпки (един хронон = една стъпка).

За една стъпка може да се премести нещо от една клетка в съседна до нея, като се движе хоризонтално или вертикално със само една клетка. При достигане на края на полето, се преминава от другия му край. Тоест ако отидем нагоре и сме на ръба, ще отидем в долната част и обратното. Аналогично за ляво и дясно.

По-формалне модел: Ако имаме редове от 1 до R и колони от 1 до C, където R и C са естествени числа. Ако се предвижим от ред 1 нагоре, ще се преместим на ред R. Ако се предвижим от ред R надолу, ще се преместим на ред 1. Ако се предвижим от колона 1 ляво, ще се преместим на колона C. Ако се предвидим от колона C надясно, ще се преместим на колона 1. За всяка друга ред или колона просто ще добавим плюс или минус 1 преместване.

Това правило кара света да опише торус или донът, като в оригиналната статия се описва „Планетата Wa-Tor... е под формата на торус, поничка, покрита изцяло с вода.“

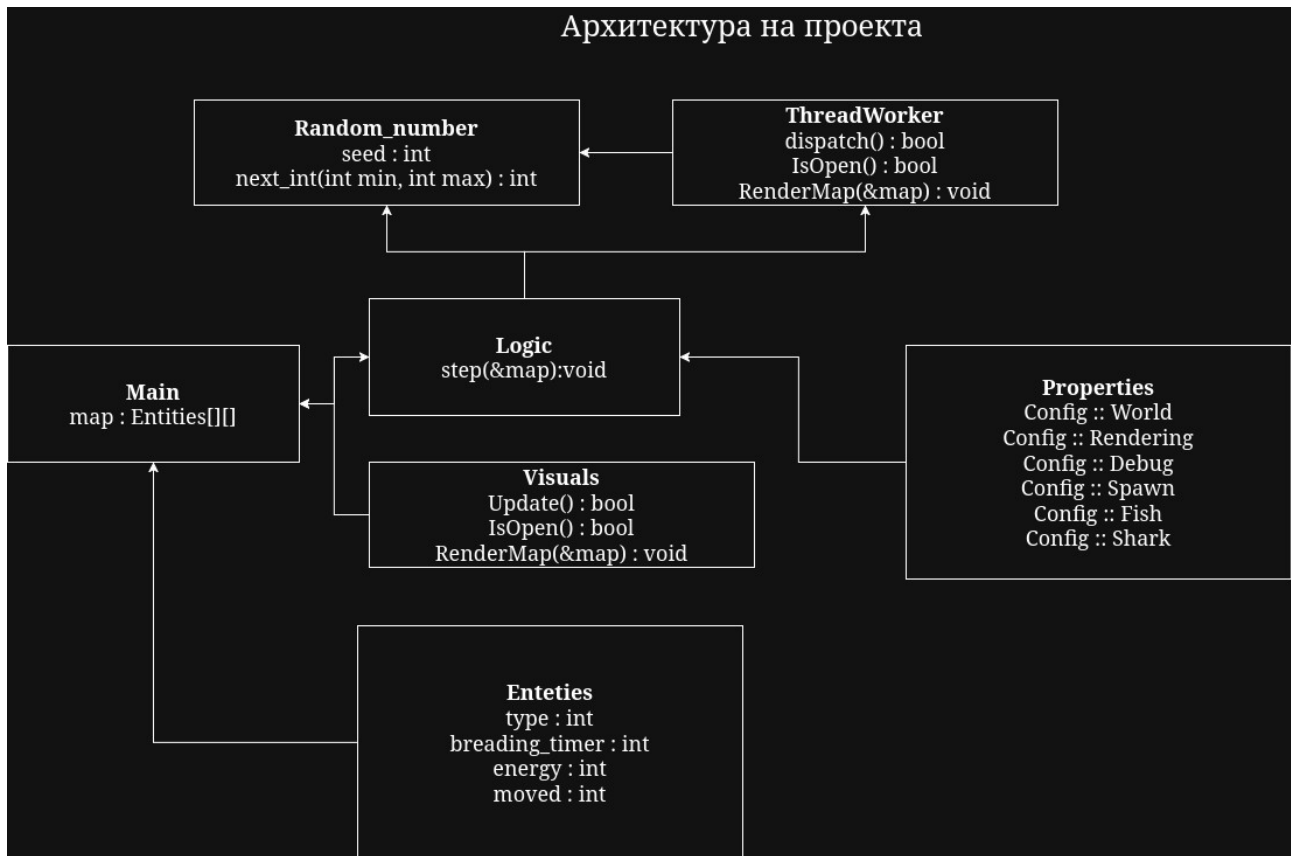
Рибите и акулите, ще наричаме още агенти, спазват тези основни правила за предвижване. Те също добавят нови правила за живеене и предвижване, като има параметри, с които може да променяме правилата на симулацията.

Структура на проекта

Проекта е съставен от 6 различни обекта.

- Visuals – визуализация на площта
- Entities – съхраняване на информация за рибите и акулите

- Random Numbers – генератор на случайни положителни цели числа
- Thread Worker – Система за нишки и обработване на задачи
- Logic – Основна логика на симулацията
- Properties – Параметри за симулацията, визуализацията и изходните данни



Агенти в симулацията

За всяко правило, до което пиеше „(конфигурируемо)“, има параметър в *parameters.hpp*

Риба

- За една стъпка, рибата се мести случайно в съседна празна клетка. Ако няма такава си остава на същата клетка.
- Ако рибата оцелее дади стъпки, тя може да се размножи. Това става като се премести в съседна клетка, а нова риба се появява на старото и място. Започват да броят стъпките за размножаването от 0. Ако рибата не може да се премести, не се размножава.
- Рибите не умират освен ако не са изядени от акула.

Акула

- За една стъпка, акулата се премества до съседна клетка с риба, при което я изяжда. Ако няма риби наоколо се мести до празна клетка. Ако няма празна клетка, остава на същото място.
- При старт се започва с определен брой енергия. (конфигурируемо)
- На всяка стъпка, акулата губи енергия. (конфигурируемо)
- При достигане на нула енергия, акулата умира.
- Ако акулата изяде риба, получава енергия. (конфигурируемо)
- Ако акулата достигне даден брой енергия, тя може да се размножи. Новата и старата акула започват с първоначална енергия като при симулацията. (конфигурируемо)

Алгоритъм

Симулацията изпълнява SPMD (Single program, multiple data) като се възползва от нишки. Single program, защото един тип програма се изпълнява, това ще бъде нашия алгоритъм за всяка нишка. Multiple data е защото разделяме полето на няколко части от един и същ тип. Тези части накрая се връщат като една голяма матрица. Броя на нишките се избира от глобален файл с параметри *properties.hpp*. Полето се разделя на равни сектори спрямо броя на нишките. Всеки сектор се обработва от една нишка.

Основната логика се намира в *1b-logic.hpp*. Процеса е следния

Извикване на нишките

Изчакване на нишките да приключат

Изчисление на време за изпълнение

Всяка нишка изпълнява следния код:

Избиране на начин на обхождане спрямо четност на времето

Обхождане на редове:

Заклучване на мютекс ако обхождаме бариерно локацията

Обхождане на всяка колона от реда

Ако е риба

местим риба

Ако е акула

местим акула

Алгоритъма има статична декомпозиция, всеки сектор е раздаден на дадена нишка. Това позволява да има детерминираност...

Сектори

Секторите представляват определен брой редове, без прекъсване между редовете или в редовете. Това дава възможност на по-проста логика и ясно разделение между нишките.

За избягване на специфични повторения и да изглежда по-органично движението на агентите, е имплементирано променяне на реда на обработка на клетките. Това се определя спрямо стъпката и нейната четност при mod 4.

Даден сектор се обработва по различни начини. При четна стъпка се обработва отгоре-надолу, а при нечетна, отдолу-нагоре. Редуват се обработване отляво-надясно и отдясно-наляво. Цикъла на изглежда по следния начин:

1. Отгоре-надолу, отляво-надясно
2. Отдолу-нагоре, отляво-надясно
3. Отгоре-надолу, отдясно-наляво
4. Отдолу-нагоре, отдясно-наляво

Благодарение на това, нашата система изглежда много по-случайна. Избягват се повторения и образуване на специфични движения. Същата симулация с стандартно движение отгоре-надолу, отляво-надясно създава ефект на изплуващи балончета, което не изглежда естествено.

Синхронизация

Основен проблем при работата с нишки е синхронизацията. В този проект нишките си комуникират със техните съседни (нагоре и надолу). За избягването на условия за състезаване (race condition) се използват Mutex системи за заключване на границата между две нишки за кратко време. Това осигурява възможност на една нишка да работи без да променя данните случайно на друга.

Тези бариери покрива общо 4 реда, 2 от едната нишка и 2 от другата. При работа на ръба на даден сектор, една нишка може да трябва да прехвърли една клетка в друг сектор. При това този сектор не трябва да модифицира реда, на който нашата нишка работи. Разбира се това не е перфектно решение, защото заключваме цял ред на друга нишка, дори ако не прехвърляме клетки или прехвърлим много малко, но така кода е доста по-просто, има по-малко mutex заключвания и знаем, че ако една нишка приключи и освободи реда, другата може да работи на него без да трябва да се върне другата или да чака.

Заключваме 2 реда, защото ако предпоследната редица може да прехвърли клетка към последната редица, а съседна нишка да направи същото, при което ще имаме колизия.

За да се избегне прехвърляне и пренаписване на нови клетки една върху друга, всяка клетка има броят, който гледа четността на стъпката. Така ако една клетка има същата четност като времето, тогава тя вече е била променена и е забранено да се променя наново.

Случайности

При избиране на случайности се използва отделен клас и проста математика. Прилага се код за случайност на симулацията (seed), който се намира в *properties.hpp*. Спрямо този код се създават случайни числа. Алгоритъмът е базиран на xorshift. За целите на този проект е имплементирано само `int next_int(int min, int max)` и се използва само за положителни числа. Всяка нишка получава собствен клас за случайни числа със специфичен код за случайност състваен от номера на нишката и оригиналния код за случайност.

Този подход осигурява, че всяко пускане на симулацията без промяна на параметрите ще върне една и съща симулация. Разбира се времето на изпълнение може да е различно зависимост от компютъра, ресурсите и много други фактори.

Параметри

Програмата предоставя различни параметри в няколко категории:

- World - параметри за размер на прозореца, визуализация, брой стъпки и брой нишки. Ако броя стъпки е 0, симулацията няма да спре и ще работи докато не бъде прекъсната от потребителя или друг процес.
- Rendering - Допълнителни параметри за визуализацията, цвят на агентите и кадри в секунда.
- Debug - Настройки за изходните данни, като дали да записва във файл, дали да брой животните в конзолата и дали да направи няколко симулации с изходни параметри.
- Spawn - Колко риби и акули да има в симулацията при стартиране.
- Fish - правила за рибите: таймер за размножаване.
- Sharks - правила за акулите: Започваща енергия, колко енергия да получи при хранене, губене на енергия при стъпка, нужна енергия за размножаване.

Визуализация

За визуализацията се използва библиотеката SFML. При отворен прозорец, може да се пазира симулацията чрез Space бутона, след което да се използва стрелка надясно, за продължение с една стъпка.

Визуализацията не зависи от размера на полето, тя се адаптира спрямо зададена резолюцията на екрана. При поле 100x100 и резолюция 1000x1000, една клетка ще бъде 10x10 пиксела.

Резолюцията не се променя спрямо размера на прозореца, а спрямо параметрите в *World*. (вижте раздел [Параметри](#))

Резултати

Източници